Exercise 1: First Conversations

Session: 1 — First Steps & Core Concepts
Duration: 30 minutes
Project: "BookStore API" — a Kotlin REST API using only the JDK HTTP server and JDBC (no frameworks). Everyone clones the same repo.

Goal

You'll have your first conversations with Claude in the IDE: explain code, find a bug, change code, say no to a change, write a test, refactor.

After each task you check how Claude got to its answer. Which files did Claude open? Which tools did it use? How many steps did it take before it answered? That is the second goal of this exercise: seeing how Claude works, not only what it says.

A few words we'll use

- **Turn**: one message from you, plus everything Claude does until it answers.
- **Cycle**: one call to the model inside a turn. One turn can contain many cycles.
- **Tool call**: one action Claude takes inside a turn. It searches the project, reads a file, edits a file, or runs a command. You see each one as a short line while Claude works.
- **Context**: everything Claude can see right now. Your messages, the code you selected, the files it read, the output of commands. `/clear` empties it.
- **Trace**: the live pane that `/xtrace tools model pane` opens next to the conversation. It shows the current turn while it runs: one row per tool call and one per cycle, with the name, what it looked at, how it went, and how long it took. Under the rows it totals the turn. It keeps every turn since the last `/clear`, one at a time.

How every task works

Open the trace once, before task 1:

```
/xtrace tools model pane
```

It opens next to the conversation, filtered to tool calls and cycles, and stays open through every task. `/clear` empties it; it does not close it.

Same four steps every time:

1. **Ask**. Send the prompt from the task. Select code first when the task says so.
2. **Watch**. While Claude works, the trace adds one row per tool call and one `model` row per cycle, in order. Which file does Claude open first? Does it run anything? How many cycles does it take?
3. **Read**. Read the answer. Does it match what you know? Does it match what you watched?
4. **Note**. Write down the number the task asks for. The tool-call count is the number of `tools` rows in the turn; the `model` rows are the cycles. You need these numbers in the closing round at the end.

Did the task take more than one turn? The trace shows one turn at a time. Press ctrl+x tab to give it your keys, then `p` for the previous turn, `n` for the next, `l` for the latest. Esc gives the keys back.

> **The loop, in short: Ask → Watch → Read → Note.** Every task below uses it. Come back here when you lose track.

About training mode: `CLAUDE.md` keeps this project in training mode. When you ask about a bug or a fix, Claude asks you one leading question before it answers. Answer it. That is part of the exercise. Stuck or short on time? Just say "*just tell me*". That is allowed.

Tasks

1. Explain, with and without a selection

Open `src/main/kotlin/bookstore/store/BookStore.kt` and select the whole `search()` function. Ask the question below. Its first line is the experiment prefix: a fixed line that stops training mode from steering the run. Both prompts in this task start with it, so the two runs stay comparable.

```
[Exercise 1 experiment — execute directly, no leading questions.]
Explain what this function does step by step
```

Read the answer. Did Claude say that the loop runs one extra query per book? Note the number of tool calls the trace shows.

Now predict, in writing, one sentence: how many tool calls will Claude need when it cannot see your selection?

Run `/clear`. Do not select anything. Ask:

```
[Exercise 1 experiment — execute directly, no leading questions.]
Explain step by step what search() in the book store does
```

Compare the two counts with your prediction.

Two notes. Your selection goes into the context directly, so Claude can answer without opening anything. Without it, Claude has to search the project first, and the trace shows you how.

Done when: you have both tool-call counts written next to your prediction.

2. Find a bug

Run `/clear`, so the trace shows this task only.

Open `src/main/kotlin/bookstore/handler/ReviewHandler.kt` and select `createReview()`. Before you ask, write one sentence: which other file will Claude open to answer?

Ask:

```
Is there a bug in this function?
```

Training mode applies here. Answer the leading question, then read the answer. That took more than one turn, and the trace shows the last one. Press ctrl+x tab, then `p` to step back a turn. Did Claude open the file you predicted? In which turn?

Done when: Claude named a bug, and you can say in one sentence what `createReview()` fails to check.

3. Add validation, then say no

Run `/clear` again, then ask:

```
Add input validation to createReview: rating must be 1-5, review_text must be between 10
and 500 characters
```

Training mode may ask a leading question here too. Answer it, then look at where Claude put the checks. Inside the handler? In a separate function? In a data-class init block? Now say no:

```
No, extract the validation into a separate validateReview() function instead
```

Saying no is normal. Claude does not mind, and you decide what goes into your code.

More than one turn again, so step back through them with `p`. Three things to look for: which tool changed the file, in which turn, and what Claude ran to check its own change. If it ran nothing, note that. Nobody asked it to.

Done when: `validateReview()` exists in `src/main/kotlin/bookstore/handler/ReviewHandler.kt`, `createReview()` calls it, and `./gradlew build` reports no errors.

4. Write a test

Ask:

```
Write a test class for validateReview() with one JUnit @Test per case covering: rating 0, rating 6, text of 9 characters, text of 501 characters, and one valid review. Put it in src/test/kotlin/bookstore/handler/ValidateReviewTest.kt, use kotlin.test assertions, and name the class ValidateReviewTest
```

Run `./gradlew test` yourself. Other tests in this package fail on purpose. Session 2 deals with them. Look only at the five tests.

Look at the trace. Did Claude run the test itself, or did you? How many tool calls came before its first edit?

Done when: `./gradlew test` shows five passing tests in `ValidateReviewTest`.

5. Refactor

Open `src/main/kotlin/bookstore/Server.kt`. The route list is flat and in no particular order. Ask:

```
Refactor Server.kt to group route registrations by resource using helper functions instead of one long list of route() calls
```

Compare before and after with `git diff src/main/kotlin/bookstore/Server.kt`. Then run `./gradlew build`.

In the trace, how many files did Claude read before it edited `Server.kt`? Which of those reads were needed for this change?

Done when: `Server.kt` has one helper per resource (books, reviews, authors), and `./gradlew build` reports no errors.

6. Reset, then ask a big question

Run `/clear`. The trace empties: the context is empty. Everything from tasks 1 to 5 is gone for Claude, not for you.

Now ask:

```
What HTTP status codes does this API return, and which ones violate REST conventions?
```

Claude started from nothing this time. Count in the trace the files it opened to answer.

Done when: you noted two numbers: files opened, and status-code issues found.

Bonus (only if time remains)

- Ask Claude the task 6 question again, in the same conversation. Then press `p` in the trace to see the earlier turn. Fewer tool calls the second time? Say why in one sentence.
- Run `/context`. How much of the context window did two questions cost?

Closing round

The trainer calls on people at random. Have your numbers ready: how many tool calls with the selection, and how many without? Which file did Claude open first for the bug, and did your prediction hold? Did Claude run your test on its own, or did you have to? How many files for the status-code question, and how many issues came out? Close with **one take-away** you'd give someone who skipped this session.